

MODULE2

UnitTestFrameworks

Name	Ayush Kar
Program/Year	B.Tech CSIT/4thYear
Institution	InstituteofEngineering&Management, Kolkata
EnrollmentNo	12023002023011
Experiments	Experiments2.1–2.3
Date	17 th September

Index—Module2Experiments

Exp. No.	ExperimentTitle	Status
2.1	UnitTest – First Test Case, Setup/Teardown,Assertions andTestSuite	Completed
2.2	PyTestFramework–Fixtures, Assertions,MultipleTestsand HTML Reporting	Completed
2.3	PageObjectModel–Page Classes, Base Class, Test ClassandCSVData	Completed

Experiment 2.1 : Unit Test Framework– First Test Case, Setup/Teardown, Assertions and Test Suite

1. Problem Statement

The task is to create a basic Python unit test using the unittest framework. The test should open the SauceDemo login page, verify the page title, perform a valid login, use assertions to check the expected result, and close the browser after each test. The experiment also demonstrates setup, teardown, and running multiple test methods as a test suite.

2. Objective

The aim is to understand the structure of Python's unittest framework and practise test cases, assertions, setup and teardown methods, and a test suite using Selenium WebDriver.

3. Tools, Software & Concepts Used

- Python 3.x
- Selenium 4
- unittest – Python's built-in unit testing framework
- Google Chrome browser with Selenium Manager
- VS Code / PyCharm IDE
- setUp() and tearDown() methods
- assertEquals() and assertIn() assertions
- unittest.TestSuite and TextTestRunner
- SauceDemo: <https://www.saucedemo.com/>

4. Source Code / Implementation Steps

```
import unittest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

class TestSauceDemo(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.get("https://www.saucedemo.com/")
        time.sleep(3)
        print("Browser opened successfully")

    def test_title(self):
        self.assertEqual(self.driver.title, "Swag Labs")
        print("Title test passed")
        time.sleep(2)

    def test_login(self):
        self.driver.find_element(By.ID, "user-name").send_keys("standard_user")
        self.driver.find_element(By.NAME, "password").send_keys("secret_sauce")
        self.driver.find_element(By.ID, "login-button").click()
        time.sleep(3)

        self.assertIn("inventory.html", self.driver.current_url)
        print("Login test passed")
        print("Current URL:", self.driver.current_url)
        time.sleep(3)

    def tearDown(self):
```

```

        time.sleep(2)
        self.driver.quit()
        print("Browserclosed")

if name=="main": unittest.main(verbosity=2)

```

5. Output

5.1 TERMINALOUTPUT

```

C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe "C:/Program Files/JetBrains/PyCharm 2026.2.1/plugins/python-ce/helpers/py
Testing started at 12:83 ...
Launching unittests with arguments python -m unittest C:\Users\debas\PycharmProjects\PythonProject\exp2.py --quiet in C:\Users\debas\Pychar

Browser opened successfully
Login test passed
Current URL: https://www.saucedemo.com/inventory.html
Browser closed
Browser opened successfully
Title test passed
Browser closed

Ran 2 tests in 29.923s

OK

Process finished with exit code 0

```

Fig2.1.1:Pycharmterminaloutputshowingthe testexecution and result.

5.2 SELENIUM/TESTOUTPUT

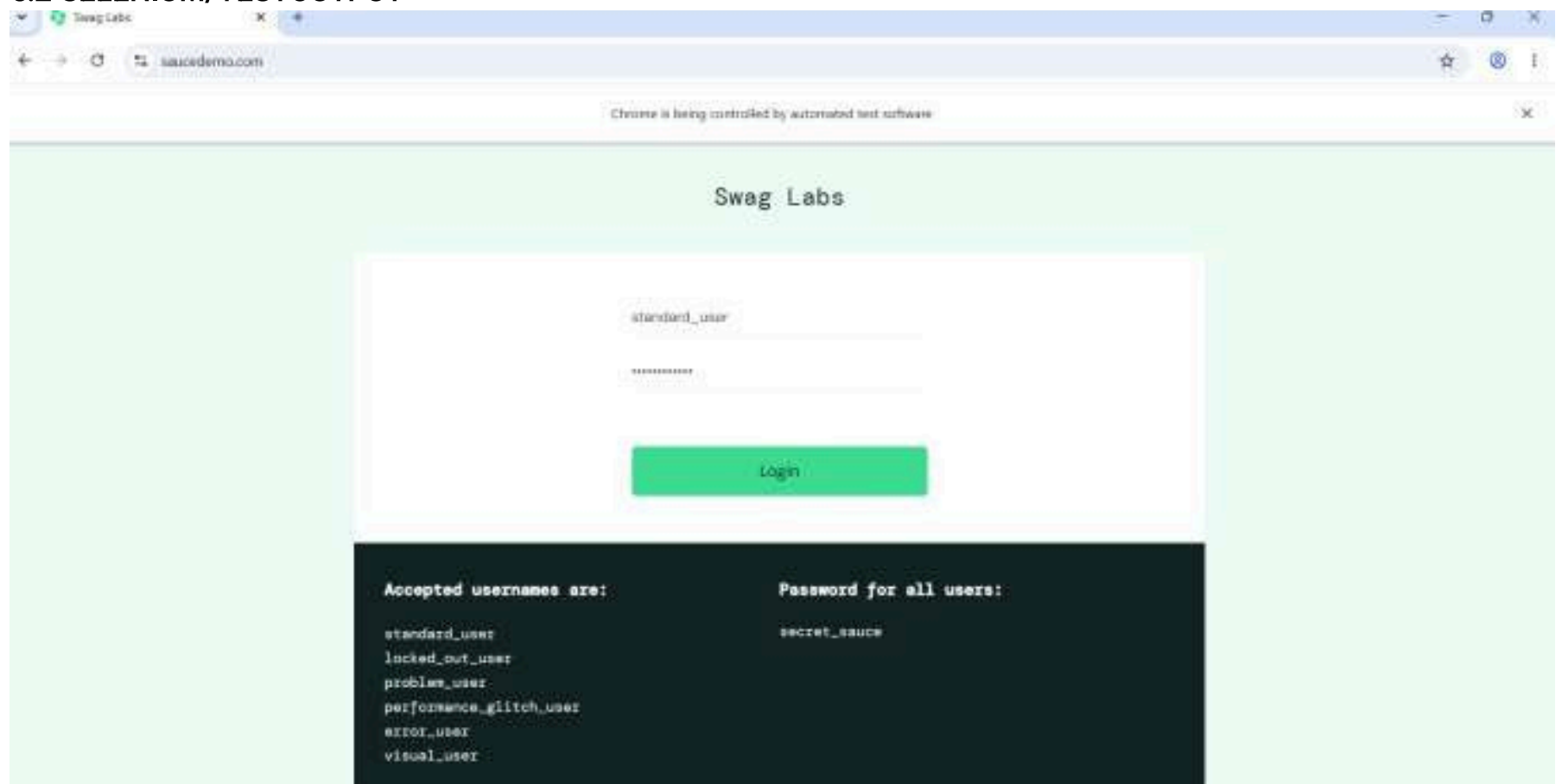


Fig2.1.2:SeleniumbrowserShowinglogging

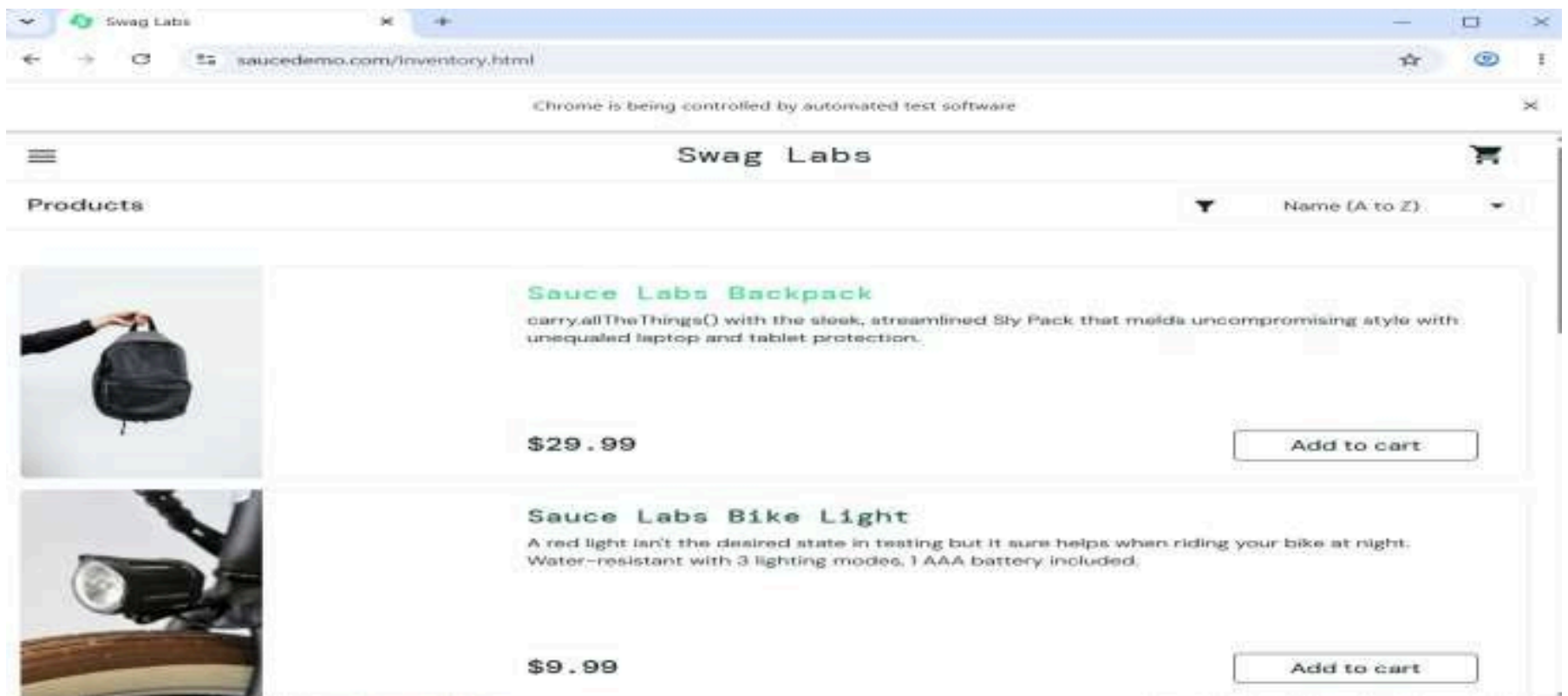


Fig2.1.3:SauceDemopage/successfullogininvisibleinChrome.

6. Result, Observation & Conclusion

Result: The unittest program executed the page-title and valid-login test cases successfully.

Observation: The terminal displayed the individual test results and success messages, while the Selenium browser showed the SauceDemo page and successful login. The `setUp()` method opened the browser before each test and `tearDown()` closed it after each test.

Conclusion: The terminal and Selenium outputs confirmed that the unittest framework worked correctly with Selenium. I learned how test methods, assertions, `setUp()`, `tearDown()`, `TestSuite` and `TextTestRunner` can be combined to organise automated tests.

Experiment2.2 : PyTest Framework– Fixtures,Assertions,MultipleTests and HTML Reporting

1. ProblemStatement

The task is to run Selenium tests using the PyTest framework. The script should use a fixture for browser initialization and cleanup, perform a page-title test and a valid-login test, and run the test file from the PyCharm terminal. An HTML report can also be generated to record the execution results.

2. Objective

The aim is to understand PyTest test-file and test-function naming conventions, fixtures, assertions, multiple test cases and HTML reporting for Selenium automation.

3. Tools,Software&ConceptsUsed

- Python3.x
- Selenium4
- pytest
- pytest-htmlpluginforHTMLreports
- GoogleChromebrowserwithSeleniumManager
- PyCharmterminal
- PyTestfixturefordriversetupandteardown
- assertstatementsforvalidation
- SauceDemo:<https://www.saucedemo.com/>

4. SourceCode /ImplementationSteps

```
import time
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By

@pytest.fixture def
driver():
    driver = webdriver.Chrome()
    driver.get("https://www.saucedemo.com/")
    time.sleep(3)
    print("Browser opened") yield
    driver time.sleep(2)
    driver.quit()
    print("Browser closed")

def test_title(driver):
    assert driver.title == "Swag Labs" print("Title
test passed") time.sleep(2)

def test_login(driver):
    driver.find_element(By.ID, "user-name").send_keys("standard_user")
    driver.find_element(By.ID, "password").send_keys("secret_sauce")
    driver.find_element(By.ID, "login-button").click()
    time.sleep(3)

    assert "inventory.html" in driver.current_url print("Login successful")
    print("Current URL:", driver.current_url) time.sleep(3)
```

RunfromPycharmterminal:pytest-vExperiment_2_2.py

GenerateHTMLreport:pytest-vExperiment_2_2.py--html=report.html

5. Output

5.1 TERMINALOUTPUT

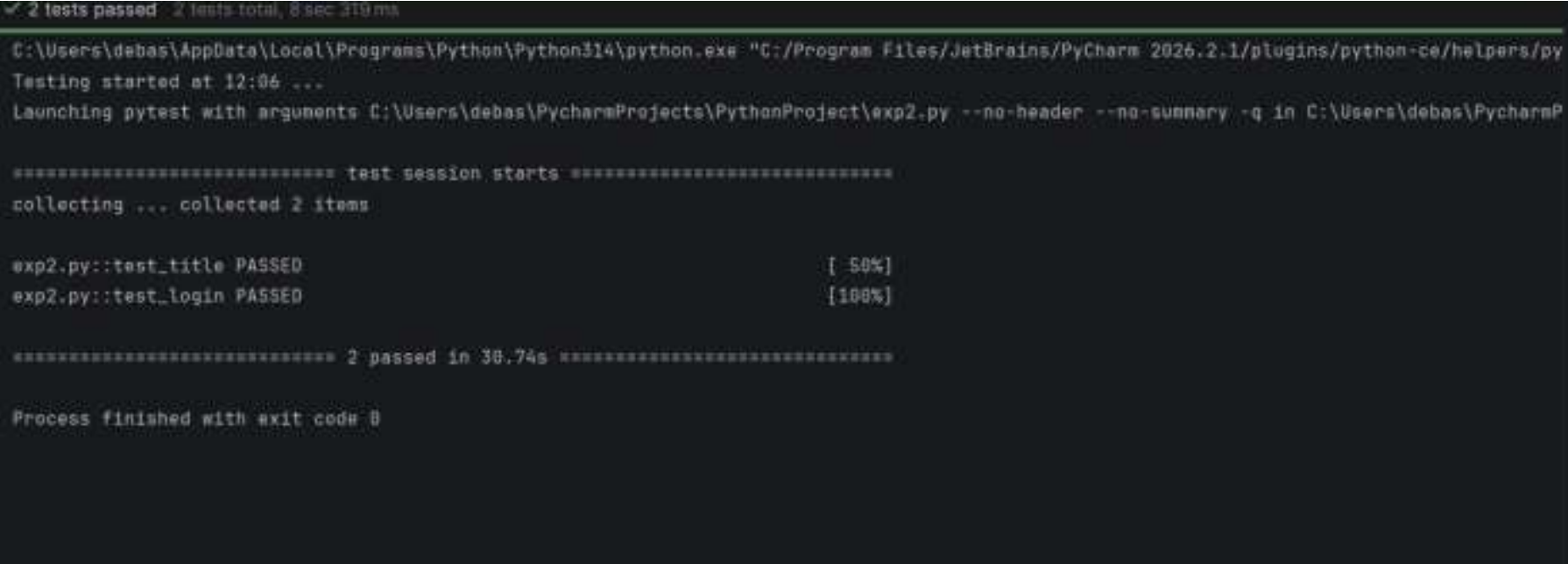


Fig2.2.1:Terminaloutputshowingthe testexecutionandresult.

5.2 SELENIUM/TESTOUTPUT

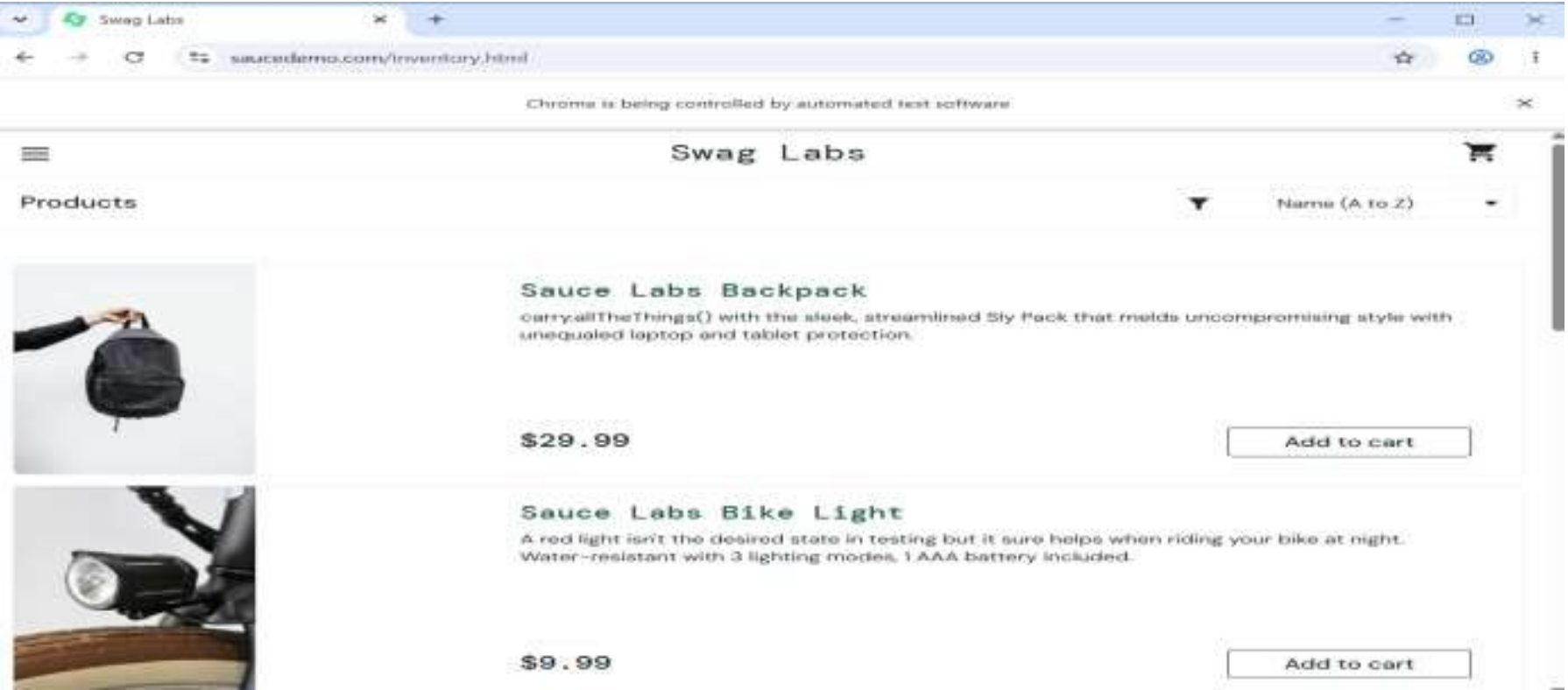


Fig2.2.2:Selenium/browseroutputshowingthecompletedautomationstep.

```
PS C:\Users\debas\PycharmProjects\PythonProject> pytest --exp2.py --html=reports/report.html
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\debas\AppData\Local\Programs\Python\Python114\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}}
rootdir: C:\Users\debas\PycharmProjects\PythonProject
plugins: html-4.2.0, metadata-3.1.1
collected 2 items

exp2.py::test_title PASSED [ 50%]
exp2.py::test_login PASSED [100%]

===== Generated html report: file:///C:/Users/debas/PycharmProjects/PythonProject/reports/report.html =====
===== 2 passed in 20.61s =====

PS C:\Users\debas\PycharmProjects\PythonProject>
```

Fig2.2.3:PyTestterminalshowingcollectedtestsandPASSEDrresults.

Experiment 2.2 – PyTest Automation Report

Selenium Web Automation | PyTest Test Execution

Report generated on 18-Sep-2025 at 03:07:34 by pytest-html v4.2.0.

Environment

Python	3.14.7
Platform	Windows-11-10.0.26200-SP0
Packages	- pytest: 9.1.1 - pluggy: 1.6.0
Plugins	- html: 4.2.0 - metadata: 3.1.1

Summary

2 tests took 00:00:49.

(Un)check the boxes to filter the results.

☐ 0 Failed ☒ 2 Passed ☐ 0 Skipped ☐ 0 Expected failures ☐ 0 Unexpected passes ☐ 0 Errors ☐ 0 Retries ☐ 0 Retried

Show all details / Hide all details

Result	Test	Duration	Links
Passed	Experiment_2_2.py::test_title	00:00:23	
Passed	Experiment_2_2.py::test_login	00:00:26	

Fig2.2.4:Generatedreport.htmlopenedinthebrowser.

6. Result,Observation&Conclusion

Result: The Selenium tests were successfully executed through PyTest and the test results were displayed in the terminal. The HTML report was generated when the pytest-html command was used.

Observation: Theterminal showedthetest names andtheir PASSED status, whiletheSelenium browser showed the page and login actions. The fixture handled browser setup and cleanup automatically, so the test functions contained only the required test steps.

Conclusion: The terminal, Selenium and HTML-report outputs confirmed that PyTest can organise Selenium tests efficiently. I learned how fixtures reduce repeated setup code and how pytest-html can provide a readable execution report.

Experiment2.3 :Page ObjectModel– Page Classes,Base Class, Test Class and CSV Data

1. ProblemStatement

Thetask is to restructure a Selenium login test using the Page Object Model (POM). The locators and page actions should be kept inside page classes, while the test class should contain the actual test and assertions. A small CSV utility is also included to demonstrate readingtest data separately from the test logic.

2. Objective

The aim is to understand the separation of base class, page class, test class and utility code and to practise a maintainable Selenium project structure using POM and external CSV test data.

3. Tools,Software&ConceptsUsed

- Python3.x
- Selenium4
- PyTest
- GoogleChromebrowserwithSeleniumManager
- PageObjectModel(POM)
- BasePageforcommondriver/pageoperations
- LoginPageforloginlocatorsandactions
- CSVmoduleforexternaltestdata
- Assertionsinthetestclass
- SauceDemo:<https://www.saucedemo.com/>

4. SourceCode /ImplementationSteps

A. testdata.csv

```
username,password
standard_user,secret_sauce
```

B. base_page.py

```
fromselenium.webdriver.common.byimportBy class

BasePage:

    definit(self,driver): self.driver =
        driver

    def open(self, url):
        self.driver.get(url)

    defget_title(self):
        returnself.driver.title
```

C. login_page.py

```
fromselenium.webdriver.common.byimportBy from
base_page import BasePage

classLoginPage(BasePage):

    username=(By.ID,"user-name")
```

```

password = (By.ID, "password")
login_button=(By.ID,"login-button")

def enter_username(self, value):
    self.driver.find_element(*self.username).send_keys(value)

def enter_password(self, value):
    self.driver.find_element(*self.password).send_keys(value)

def click_login(self): self.driver.find_element(*self.login_button).click()

def login(self,username,password):
    self.enter_username(username)
    self.enter_password(password)
    self.click_login()

```

D. test_login.py

```

import csv
import time
from selenium import webdriver
from login_page import LoginPage

def read_data():
    with open("testdata.csv") as file:
        return list(csv.DictReader(file))

def test_login():

    data = read_data()[0]

    driver=webdriver.Chrome()
    driver.get("https://www.saucedemo.com/")
    time.sleep(3)

    page = LoginPage(driver)
    page.login(data["username"],data["password"])

    time.sleep(3)

    assert "inventory.html" in driver.current_url print("Login successful")
    print("CurrentURL:",driver.current_url)

    time.sleep(5) driver.quit()
    print("Browser closed")

```

Project structure:

```

pom/
|
|--base_page.py
|--login_page.py
|--test_login.py
|--testdata.csv

```

5. Output

5.1 TERMINALOUTPUT



Fig2.3.1:Pycharmterminaloutputshowingthe testexecution and result.

5.2 SELENIUM/TESTOUTPUT

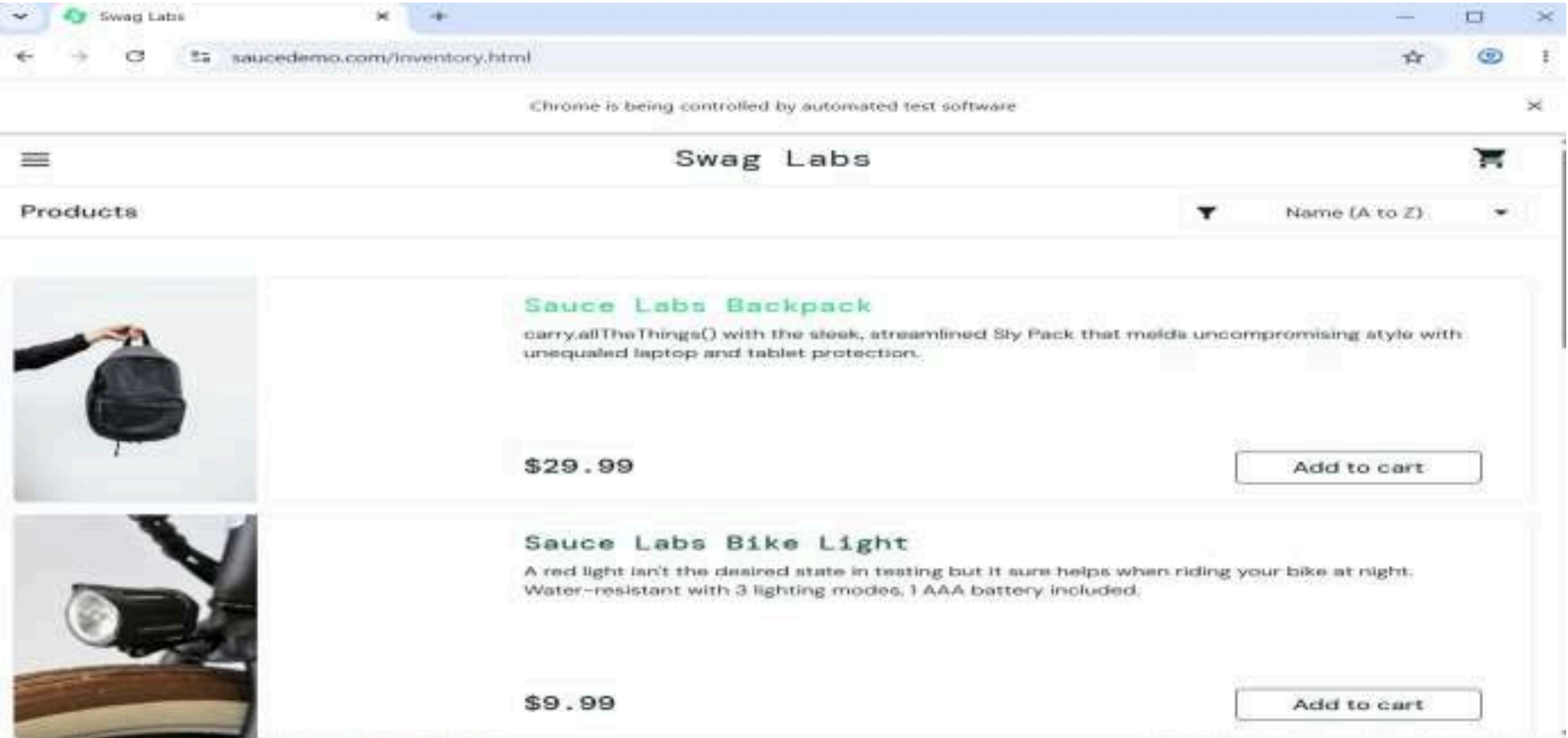


Fig2.3.2:ScreenshotshowingtheSauceDemoProductspageaftersuccessfullogin.

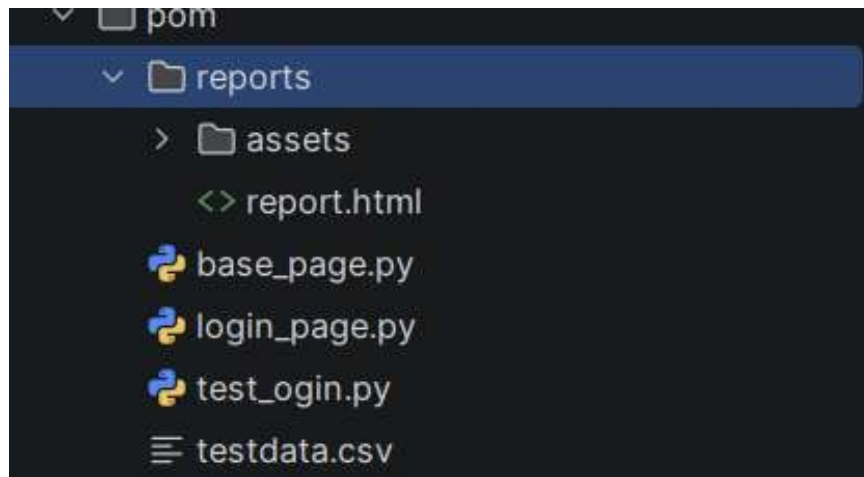


Fig2.3.3:PycharmshowingthePOMprojectfiles.

6. Result,Observation&Conclusion

Result: TheSelenium logintest was successfully reorganised using the Page Object Model, and the login data was read from a separate CSV file.

Observation:The Selenium browser showed the Sauce Demo login and inventory page, while the terminal confirmed that the POM login test passed. The page class contained the locators and UI methods, and the test class handled the test flow and assertion.

Conclusion: The terminal and Selenium outputs confirmed that the POM structure worked successfully. I learned how separating locators, page actions, test logic and test data makes Selenium automation easier to maintain and reuse.

